

Algorísmia Q2 2020–2021

Examen Final

14 de juny de 2021

Exercici 1 (2 punts). Heu de justificar totes les vostres afirmacions, en cas contrari la nota de la pregunta serà 0. Quan la pregunta sigui una qüestió (Cert o Fals?), heu de dir si l'enunciat és cert o fals abans de justificar la resposta.

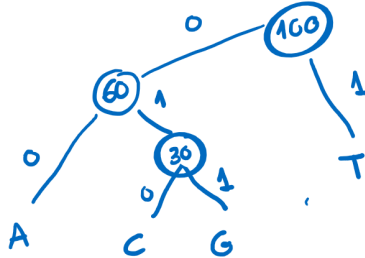
- 1.A. **(0.25 punts)** Cert o Fals? Donat un graf connex, no dirigit i amb pesos, que té com a mínim 3 vèrtexs i on tots els pesos són diferents, l'aresta amb més pes mai serà a un MST.

Falso.

Si el grafo es un árbol todas sus aristas están en el MST, incluyendo la de más peso.

- 1.B. **(0.5 punts)** Tenim una cadena de DNA sobre un alfabet $\{A, C, G, T\}$ la freqüència de cada caràcter a la cadena és, respectivament, del 30%, 20%, 10% i 40%. Doneu una codificació de Huffman per a cada caràcter.

El árbol de Huffman queda



Lo que nos da la codificación

A: 00 C: 010 G: 011 T: 1

- 1.C. **(0.5 punts)** Sigui A una taula que conté n claus, de les quals com a màxim hi ha k claus diferents (no necessàriament enters), on $k \leq \lg n$. Es pot ordenar la taula, mantenint la posició relativa inicial dels elements replicats, en temps $o(n \lg n)$?

Cierto.

Primero asociaremos a cada elemento una clave numérica entre 0 y $k - 1$. Para ello podemos realizar un insertion sort de los valores en A sin incluir duplicados en una tabla B . Para saber si una clave está o no en ya B hacemos una búsqueda dicotómica con coste $O(\lg k)$. Si no está la insertamos con coste $O(k)$, esto lo haremos como mucho k veces. Una vez ordenadas, con coste $O(n \lg k)$, podemos asignar a cada clave su posición en el orden de claves, lo que nos da una nueva clave numérica que respeta el orden inicial.

El segundo paso es ordenar A de acuerdo con las nuevas claves numéricas. Utilizando counting sort lo podemos hacer en tiempo $O(n + k)$.

El coste total es $O(n \lg k + k^2 + n + k)$, como $k \leq \lg n$, $c(n \lg k + k^2 + n + k) \leq c(n \lg \lg n + \lg n^2 + n + \lg n) = o(n \lg n)$.

- 1.D. **(0.5 punts)** Tenim un graf dirigit acíclic $G = (V, E)$ amb pesos a les arestes $w : E \rightarrow \mathbb{R}$. Donats (G, w) i $u, v \in V$, es pot obtenir un camí de u a v de pes màxim a G en temps $O(|V| + |E|)$?

Cierto.

Para obtener el camino con peso máximo podemos multiplicar por -1 los pesos y así los podemos obtener como caminos de peso mínimo. Como el grafo es acíclico, no hay ciclos con peso negativo y la distancia está definida.

Para calcular las distancias, utilizamos el algoritmo específico de cálculo de distancias mínimas en DAG's que utiliza el orden topológico y PD con coste $O(n + m)$.

- 1.E. **(0.25 punts)** Supposeu que una persona ens presenta una possible solució al problema del flux màxim a una xarxa de flux. Doneu un algorisme que en temps lineal ens determini si la solució proposada té valor màxim.

Nos basta con comprobar que no hay caminos aumentadores. Para ello construimos el grafo residual y comprobamos con un BFS si hay o no caminos de s a t . El coste total es $O(|V| + |E|)$ que es lineal en el tamaño del grafo.

Exercici 2 (2.5 punts) Sigui X un conjunt de n intervals a la recta real. Una coloració pròpia de X assigna un color a cada interval, de manera que dos intervals que se superposen tenen assignats colors diferents. Descriu i analitza un algorisme golafre eficient per obtenir el mínim nombre de colors necessaris per acolorir (amb una coloració pròpia) un conjunt d'intervals X . Podeu assumir que l'entrada està formada per dos vectors $L[1..n]$ i $R[1..n]$, representant els extrems esquerres (L) i drets (R) dels intervals a X .

Una solució:

Aquest problema és formalment equivalent al problema de la *partició d'intervals*.

Ordenem els vectors L i R de manera que estiguin en ordre creixent d'extrem esquerre:

$$L[1] \leq L[2] \leq \dots \leq L[n]$$

Suposem que hem determinat que s'han necessitat k^* colors per la coloració dels intervals 1 a $i - 1$. Més encara, hi ha $k \leq k^*$ colors aparentment en ús (per cobrir el punt $L[i] - \epsilon$). Una estructura de dades conté la informació dels k intervals (específicament els seus extrems drets) que ténen un dels k colors assignats i suposarem que podem accedir i eliminar eficientment d'aquesta estructura (un min-heap) l'interval que té R mínima. Sigui $R_{\min}$ aquest valor. Llavors si $R_{\min} \leq L[i]$ llavors el color assignat al interval corresponent es pot assignar al interval $X[i] = (L[i], R[i])$. S'elimina de l'estructura l'interval amb $R_{\min}$, que tenia assignat el color j , $1 \leq j \leq k$, i s'afegeix $X[i]$, assignat-li el color j i la seva prioritat seria $R[i]$. Si $R_{\min} > L[i]$ tots els k colors estan en ús i s'haurà d'assignar un nou color, el $k + 1$, a l'interval $X[i]$ i s'afegirà a l'estructura amb prioritat $R[i]$. Si $k^* = k$ llavors actualitzem $k^* := k + 1$. En acabar, k^* és el mínim nombre de colors necessari que s'ens demana.

El cost de l'algorisme és $O(n \log n)$ per a ordenar els n intervals per L creixent i $O(n \log n)$ per a processar els n intervals: a cada una de les n iteracions s'ha d'actualitzar el min-heap amb una inserció (la de l'interval $X[i]$ en curs) i ocasionalment amb una eliminació del mínim quan ens consta que un color queda alliberat.

A cada iteració podríem esborrar tots els intervals del min-heap amb R més petita o igual que $L[i]$ de manera que el min-heap no contingués intervals "acabats" i que ja no necessiten tenir color assignat; però és innecesari i molt més senzill esborrar a cada iteració només un interval (o cap si no és possible).

Existeixen moltes altres solucions eficients (amb cost $\Theta(n \log n)$)—però no donem justificació ni gaires detalls. Aquestes solucions alternatives no requereixen estructures de dades addicionals i un cop completada una fase d'ordenació només fan un bucle amb cost $\Theta(n)$. Per exemple podem ordenar els $2n$ extrems de menor a major i recórrer el vector, incrementant un comptador cada cop que veiem un L i decrementat cada cop que veiem un R . El valor màxim del comptador és el nombre mínim de colors demanat. Altra idea és ordenar els intervals per R creixent de manera que

$R[1] \leq R[2] \leq \dots \leq R[n]$. Llavors per a cada i , si $L[i] > R[i - k]$ ón k és el nombre màxim de colors necessari per colorejar els intervals 1 a $i - 1$ això vol dir que l'interval i -èsim no necessita un color nou, pot reutilitzar el color de l'interval $(i - k)$ -èsim. Altrament incrementem k .

© Professorat GRAU-A

Exercici 3 (2.5 punts)

Una operació habitual als laboratoris de biologia molecular és la de trossejar una cadena d'ADN, tallant-la a unes determinades posicions d'interès. En termes computacionals, s'acostuma a representar una cadena d'ADN com un mot sobre l'alfabet $\{A, C, G, T\}$.

Assumim que tallar una cadena d'ADN $c = c_1 \cdots c_n$ de longitud n en dos trossos qualssevol té cost n . Quan parlem de fer un tall a la posició p_i , el símbol d'aquella posició queda com a últim símbol del tros esquerre (és a dir, tallem darrera de p_i).

És fàcil veure que, si es vol fer una seqüència de talls a unes determinades posicions d'interès $p_1, \dots, p_k$, amb $p_i \in [1, \dots, n]$, l'ordre en què es facin aquests talls afectarà el cost total de l'operació. Preneu com a exemple el cas d'haver de trencar una cadena de longitud $n = 200$ a les posicions 20, 80, 100 i 150. Si fem els talls d'esquerra a dreta, primer tallem una cadena de 200 (a la posició 20) i ens queden dos trossos, un de 20 i un altre de 180; el segon tall el farem a la posició 80 amb cost 180, etc. El cost total serà $600 = 200 + 180 + 120 + 100$. En canvi, podríem fer el primer tall a la posició 100 (amb cost 200) i després tallar cadascun dels dos trossos ($c_1 \cdots c_{100}$ i $c_{101} \cdots c_{200}$) d'esquerra a dreta. Això ens donaria un cost de $480 = 200 + (100 + 80) + (100)$.

Doneu un algorisme de PD tal que, donada una cadena d'ADN i la seqüència de posicions a l'esquerra de les quals volem tallar la cadena, ens proporcionï la forma menys costosa de tallar-la. El vostre algorisme ha de proporcionar el millor cost possible per realitzar els talls i una estructura de dades que ens permeti esbrinar en quin ordre procedir amb els talls per tal d'aconseguir el cost òptim.

Una solució:

[NOTA: Observeu que aquest problema assembla al problema de la multiplicació d'una cadena de matrius (vist a teoria), on s'ha de decidir com parentitzar les multiplicacions.]

Anomenem P al conjunt de k posicions d'interès que determinen on s'han de realitzar els talls de la seqüència. Assumirem que P no té repeticions (no té sentit que en tingui) i que està ordenat creixentment, és a dir,

$$P = \{p_1, \dots, p_k\} \quad \text{on} \quad \forall i \in [1, k] : p_i \in [1, \dots, n], \quad \text{i} \quad \forall i \in [1, k] : p_i < p_{i+1}.$$

Subestructura òptima: Identifiquem primer la subestructura òptima del problema. Considerem una seqüència òptima $p_{\pi(1)}, p_{\pi(2)}, \dots, p_{\pi(k)}$ de realització dels talls, és a dir, la seqüència òptima en què primer es trenca la seqüència C a $c[p_{\pi(1)}]$, després a $c[p_{\pi(2)}]$, etc. Matemàticament, la funció $\pi : \{1, \dots, k\} \rightarrow \{1, \dots, k\}$ és una permutació de k elements que ens permet descriure una determinada permutació de la seqüència P de talls a realitzar.

Considerem el primer punt de tall $p_{\pi(1)}$, que trenca la cadena C en dues subcadenaes C_1 i C_2 .¹ Aleshores, la subestructura òptima és com segueix: de la subseqüència de talls

¹Sigui $c_\alpha = c[p_{\pi(1)}]$, aleshores $C_1 = [c_1, \dots, c_\alpha]$ i $C_2 = [c_\alpha + 1, \dots, c_n]$.

$p_{\pi(2)}, p_{\pi(3)} \dots, p_{\pi(k)}$ restants, els que són més petits que $p_{\pi(1)}$ trenquen C_1 de manera òptima, i els que són més grans que $p_{\pi(1)}$ trenquen C_2 de manera òptima.

Recurrència: Seguint la subestructura òptima descrita, definim $T(i, j)$ com el cost òptim de dur a terme el subconjunt ordenat de talls $\{p_i, \dots, p_j\}$ de P .

Afegirem a P dos talls addicionals: $p_0 = 0$ i $p_{k+1} = n + 1$. Aquest afegitò tècnic no modifica en res la solució obtinguda i ens facilitarà la formulació de la recurrència. Aleshores, $T(i, j)$ es calcula segons indica la següent recurrència:

$\forall i, j \in [0, k + 1] :$

$$T(i, j) = \begin{cases} 0, & \text{si } j = i \text{ o } j = i + 1 \text{ (casos base).} \\ \min_{i < t < j} \{T(i, t) + T(t, j) + (p_j - p_i)\}, & \text{altrament.} \end{cases}$$

Observeu que els casos base els defineixen dues situacions: quan $j = i$ i quan $j = i + 1$. Ambdues representen situacions en què no es realitza cap tall. En la resta de casos, el cost òptim de dur a terme els talls $\{p_i, \dots, p_j\}$ requereix buscar el punt de tall $p_t \in \{p_i, \dots, p_j\}$ que optimitza els costos dels subproblemes $\{p_i, \dots, p_t\}$ i $\{p_t, \dots, p_j\}$ generats, sumat al propi cost de fer un tall sobre el segment $\{p_i, \dots, p_j\}$ (que sempre és el mateix, independentment del punt p_t on es triï fer el tall).

Segons aquesta definició, la solució a l'objectiu del problema la computa el valor $T(0, k + 1)$ d'aquesta recurrència.

Cost: L'entrada al problema és la seqüència $C = [c_1, \dots, c_n]$ d'ADN i el conjunt $P = \{p_1, \dots, p_k\}$ de talls a realitzar, on $k = |P| \leq |C| = n$. El cost temporal de l'algoritme ve determinat pels $\Theta(k^2)$ subproblemes que s'han de resoldre, i pel cost $\Theta(k)$ que necessita cadascun d'aquests subproblemes pel fet d'haver de cercar el punt de tall. Per tant, el cost temporal total és $\Theta(k^3)$. El cost espacial és $\Theta(k^2)$, necessari per tabular el $T(i, j)$ de cada subproblema.

Construcció de la solució òptima: La resolució de $T(0, k + 1)$ ens permet obtenir el cost de la millor solució que particiona la seqüència. Si adicionalment volem també especificar quina és exactament la seqüència de talls que correspon a la solució òptima, necessitem guardar, per a cada $T(i, j)$, quin ha estat el punt de tall t que ha produït el mínim a la recurrència. Això és molt fàcil de implementar fent que cada posició de la taula sigui una tupla on es guardi també aquest valor.

Una vegada s'ha computat $T(0, k + 1)$, podem anar reconstruint la seqüència òptima de talls accedint recursivament als valors òptims corresponents a les dues solucions òptimes parcials (esquerra i dreta) que defineixen cada tall. Observeu que, en realitat, aquesta seqüència de talls ens està representant una mena d'arbre binari on es van veient els trossos que van produïnt cadascun dels talls segons l'ordre resultant. Si, per

exemple, volem descriure com es van produïnt els talls per nivells, podríem implementar un algorisme com el següent:

funció IMPRIMEIX-TALLS-ÒPTIMS (T, k)

```
Q.PUSH((0, k + 1))
mentre no buida Q fer
    ( $i, j$ )  $\leftarrow$  Q.POP()
     $t \leftarrow T(i, j).$ SECOND()
    PRINT( $t$ )
    si  $t - i > 1$  aleshores
        Q.PUSH(( $i, t$ ))
    fi si
    si  $j - t > 1$  aleshores
        Q.PUSH(( $t, j$ ))
    fi si
fi mentre
fi funció
```

▷ $T(0, k + 1)$ té la solució òptima al problema

▷ Cua de trossos tallats

▷ Punt de tall òptim del tros en tractament

▷ Encuem el tros esquerre

▷ Encuem el tros dret

Aquesta funció té cost $\Theta(k)$.

Exercici 4 (3 punts). El RACC està interessat a trobar un mecanisme per poder suggerir als seus clients rutes alternatives per fugir dels embussaments de trànsit al centre de Barcelona. El Servei Català de Trànsit (SCT) els hi ha proporcionat un model simplificat del centre de Barcelona, que està format per un graf $G = (V, E)$ amb un vèrtex per cada cruïlla, i una aresta per cada parell de cruïlles connectades perquè hi ha un carrer que les uneix. A més coneixen el subconjunt de cruïlles $B \subseteq V$ que donen accés a carrers que surten del centre de la ciutat.

Dels telèfons mòbils dels clients, el RACC pot obtenir informació sobre el conjunt $C \subseteq V$ format per les cruïlles més rellevants als llocs on es troben els cotxes dels seus clients. L'SCT proporciona també el conjunt de trams de carrers $E' \subseteq E$ amb un nivell baix de saturació de trànsit i el conjunt $D \subseteq V$ de cruïlles crítiques que vol evitar.

El RACC vol determinar si és possible trobar un conjunt de camins disjunts (que no comparteixen arestes) que faci servir només trams amb baix nivell de saturació tals que, per cada cruïlla $c \in C$ tinguem un camí que comença a c i que acaba a alguna de les cruïlles de B . A més cal que una cruïlla crítica (de D) aparegui com a molt a 2 d'aquests camins.

Doneu un algorisme tan eficient com pugueu que, donats G , E' , B , C i D , determini si és possible obtenir els camins requerits i que, en cas que ho sigui, retorni, per a cada $c \in C$, el camí que comença a c i acaba a alguna cruïlla de B .

Una solució:

Una solución: Lo resolveremos mediante un problema de flujo con restricciones. Tenemos restricciones en el flujo en algunos nodos y en los inicios y finales de los caminos. Además solo podemos utilizar las aristas en E' con capacidad 1 para garantizar que no se usan más de una vez. Para limitar la capacidad de los cruces en D desdoblaremos los vértices correspondientes en dos, uno recibirá las aristas de entrada y del otro saldrán las de salida, la arista conectando las dos copias tendrá capacidad 2.

La entrada es $G = (V, E)$, E' , y C, B, D ,. A partir de G consideramos la siguiente red de flujo. La red $\mathcal{N}$ tiene

- Nodos: $s, t, V, D' = \{d' \mid d \in D\}$ una copia de D ,
- Aristas y capacidades:

$\{(s, c) \mid c \in C\}$	capacidad 1
$\{(b, t) \mid b \in B\}$	capacidad $ C $
$\{(d, d') \mid d \in D\}$	capacidad 2
$\{(d', u) \mid d \in D, u \in V - D, (d, u) \in E'\}$	capacidad 1
$\{(u, d) \mid d \in D, u \in V - D, (u, d) \in E'\}$	capacidad 1
$\{(u, v) \mid u \notin D, v \notin D, (u, v) \in E'\}$	capacidad 1

Un camino de s a t tiene la forma $s \rightarrow c \rightarrow \dots \rightarrow d \rightarrow t$, si transporta una unidad de flujo interpretaremos que hay una ruta desde $c \in C$ a $d \in D$. La capacidad de las aristas aseguran que, en un flujo con valor máximo, en ningún caso se utiliza una arista de E' más de una vez ni se supera el límite de tráfico por los vértices de D .

Para resolver el problema primero obtendremos un flujo con valor máximo en la red, esto nos proporcionará el número máximo de caminos que cumplen las restricciones de seguridad. Si este valor coincide con $|C|$ el problema tiene solución. En este caso podemos extraer los caminos utilizando el grafo formado por las aristas con flujo 1. Con un BFS podemos extraer un camino, restar una unidad de flujo en las aristas de este camino. Repetimos el proceso hasta extraer los $|C|$ caminos.

Algoritmo:

```
Construir  $\mathcal{N}$ 
 $F = \text{MaxFlow}(\mathcal{N})$ 
if ( $|F| < |C|$ ) then
    return No hay solución
else
    Obtener caminos desde  $C$ 
end if
```

El número de vértices en la red es $N = |V| + |D| + 2$, el número de aristas es $M = |M'| + |C| + |D| + |B|$. Como el flujo máximo está acotado por $|C|$, utilizando el análisis de FF el coste es $O(|C|(N + M)) = O(m^2n)$. La última parte tiene también este coste.